Adaptive lattice graph for selected Eisenstein digit points

Question. Can selected 31+32τ digit points share projective synthesis work through an addition graph, and does the measured gain survive fixture replay?

Scope. This engineering study is on SEC 2 secp256k1, $E: y^2 = x^3 + 7$ over $p = 2^{256} - 2^{32} - 977$, subgroup order n=FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141. The frozen 12,288 scalar/point fixture has SHA-256 a1c4964f174e36ea26f6a1897128f651a2ce7251e4bf4ffd532ff09221e721d3. No curve, isogeny, or new scalar rule is proposed.

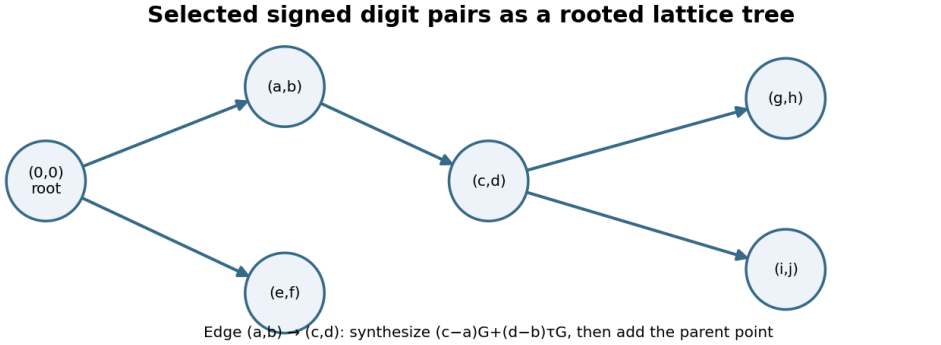


Figure 1: Adaptive lattice graph construction

Method and source review. The hash-guarded native overlay starts from the scalar-local projective digit set and uses Prim's algorithm on its signed lattice pairs. An edge from parent `(a,b)` to target `(c,d)` synthesizes `(c−a)G+(d−b)τG` from an eight-bit shared dyadic basis, then adds the parent's already constructed point. The root `(0,0)` remains an available direct edge to every target. In the declared operation-count model, a root edge costs the number of set bits minus one, while a non-root edge charges the displacement synthesis and parent addition. Thus the minimum spanning tree is no worse than the direct-root star in that model. The model omits graph-search cost and treats group additions uniformly; the measured interval includes scalar-local preparation and evaluation, so wall-clock results include the graph search.

The graph selection and construction depend on each scalar's selected digits and pair distances. This implementation is **not constant-time** and must not be used for secret-scalar multiplication. No side-channel review or peak-memory measurement is claimed.

Lean scope. The pinned Lean module proves that adding the pair-lattice displacement reconstructs a target and that any evaluation map preserving pair addition transports that identity to a point addition. It also checks the direct root displacement. The module and dependencies were rebuilt with Lean 4.34.1; the new dated proof receipt matches current source hashes and records `#print axioms`. The graph lemmas depend only on standard `propext` and `Quot.sound`; `lean4checker --fresh` was unavailable. This does not prove that Rust's dyadic builder or `add_projective` implements the abstract maps for every secp256k1 point; the finite native replays are scoped checks of that bridge.

Matched Tier 1 panels. The one-scalar and 128-point manifests compare the graph path with independent projective synthesis on identical inputs, CPU 40 on RunPod pod `thyvmdqmzsz5ik`, 16 alternating repetitions, and adjacent reference/reference A/A controls. Each completed 96 arms and 48 pairs. Rust `Instant` covers scalar-local preparation and Horner evaluation; process launch, fixture loading, and external output checking are outside the interval. The rules, launches, and raw archive identify job 29a9d3b4552b4d809b9d4462738cb291. The corresponding 128-point launch and raw archive identify job b5dcc32046b34c7faf6ad6249d0c0888; the provenance archive retains exact sources and binaries.

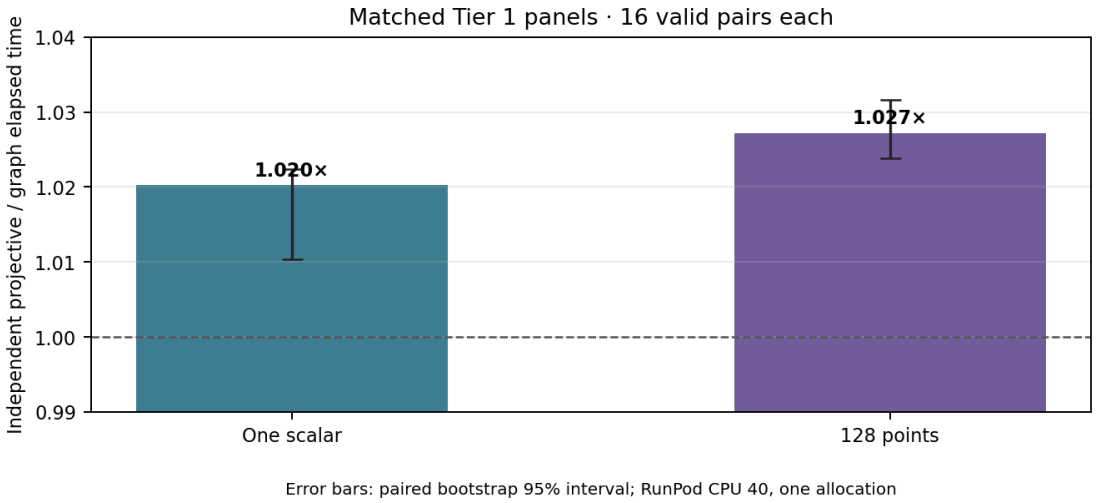


Figure 2: Matched Tier 1 graph ratios

Frozen workload	Independent projective mean	Graph mean	Qualified paired median	Paired bootstrap 95% interval
One scalar	0.543 ms	0.530 ms	1.020×	[1.010, 1.022]
128 points	39.983 ms	38.859 ms	1.027×	[1.024, 1.032]

The independent auditor confirmed source, binary, provenance, and raw hashes; exact commands and outputs; pair records; independent multiplication of measured fixture points; stable artifacts; and all A/A controls. Both panels were qualified with no invalid arms or A/A failures. These are small Tier 1 gains on one allocation, not general guarantees.

Full-fixture replay. Job `91167c97ca044280a925af245f13fa82` used two identical graph arms from the frozen manifest, launch, and raw archive. Each arm verified all 12,288 frozen outputs; the independent auditor checked the fixture, binary, raw run records, output assertions, pair ledger, and independent last-scalar replay. The arms took 3.759 s and 3.740 s. This A/A replay has a null speedup and verifies only this finite fixture.

Graph and remaining gap. The experiment index and digit-graph source record now link this report. The graph represents a scalar-multiplication implementation optimization, not a curve/isogeny route; I checked the canonical curve catalog and route records and found no graphable finding to add. The shared sparse-radix point-correctness problem remains open: the map-add Lean theorem is conditional, and neither finite replay proves all-scalar digit selection or the native group-law refinement.

The editable figure source, SVG and 180-dpi PNG files, and this report's PDF are committed together.